

PROYECTO FINAL ALGORITMOS

K.A. Rodas Santos

7690-25-4173 Universidad Mariano Gálvez

krodass7@miumg.edu.gt

2. Resumen

Mientras he ido aprendiendo a programar, me he dado cuenta de que entender lo básico es lo que realmente me ayuda a avanzar. Primero, aprender la sintaxis de lenguajes como Python o C++ me permitió empezar a escribir mis propios programas. Luego, entendí la importancia de manejar errores con excepciones para que el programa no se detenga por cualquier cosa. También aprendí a trabajar con archivos, lo cual es útil cuando necesito guardar información o leer datos.

Uno de los temas que más me ha servido es la programación orientada a objetos. Gracias a eso comprendí qué son las clases, los objetos y cómo podemos organizar mejor nuestro código usando métodos, atributos, herencia y encapsulamiento.

También he aprendido a usar diferentes estructuras de datos como listas, diccionarios, tuplas y conjuntos, que me ayudan a manejar información de forma más eficiente dependiendo de lo que necesite hacer. Finalmente, la idea de dividir el código en funciones me hizo ver lo importante que es mantener todo ordenado y facilitar el trabajo cuando el programa va creciendo.

3. Palabras Claves

“Sintaxis”, “Archivos”, “Clases”, “Funciones”

4. Desarrollo del Tema

La sintaxis de lenguajes como Python o C++ es la primera puerta de entrada al mundo de la programación. Aunque cada lenguaje tiene su estilo, al final ambos buscan lo mismo: que podamos comunicarnos con la computadora. Python, por ejemplo, es más simple y directo, mientras que C++ es más estricto y poderoso en el manejo de memoria. Aprender sus clases, objetos y la manera de construir estructuras básicas me ha ayudado a ver cómo se crean programas más completos.

Otro aspecto clave es el manejo de excepciones. Aquí es donde realmente se nota la importancia de escribir código responsable. Las excepciones permiten que un programa no se caiga ante el primer error, sino que pueda anticiparlo y reaccionar de forma adecuada. El famoso try-except en Python nos enseña que el software debe ser capaz de manejar problemas reales, como un archivo que no existe, un dato incorrecto o incluso un fallo del usuario.

Hablando de archivos, aprender a leerlos y escribirlos es casi como aprender a comunicarnos con el “mundo exterior”. Un programa que solo muestra texto en pantalla es útil, pero uno que guarda información o la recupera de un archivo ya empieza a resolver problemas reales. Desde archivos de texto simples hasta bases de datos, entender estas operaciones ha sido fundamental para pensar en proyectos más grandes.

En esta línea también están las interfaces gráficas (GUI) y el manejo de eventos. Crear una ventana, botones o controles nos permite llevar nuestras ideas más allá de la consola y hacer programas más amigables. Las GUI también nos enseñan un concepto muy importante: los eventos. Cada clic, cada tecla y cada interacción del usuario es algo que debemos escuchar y

procesar. Esto me hizo entender que la programación no siempre sigue un flujo lineal; muchas veces depende de lo que haga el usuario.

Uno de los temas más importantes es la programación orientada a objetos (OOP). Aquí es donde empiezan a cobrar sentido ideas como clases, objetos, métodos y atributos. Me gusta pensar en una clase como una plantilla, y los objetos como instancias de esa plantilla. Los métodos permiten que esos objetos hagan cosas, y los atributos guardan la información que necesitan. El encapsulamiento, por su parte, me ayudó a ver la importancia de proteger los datos, evitando que cualquier parte del programa los modifique sin control. Y la herencia es algo que realmente facilita la vida: permite crear nuevas clases basadas en otras ya existentes, ahorrando tiempo y código.

Otro pilar fundamental son las estructuras de datos básicas. Las listas, por ejemplo, son súper útiles cuando necesitamos manejar colecciones de elementos, ya sea para agregarlos, eliminarlos o buscarlos. Los diccionarios, en cambio, son perfectos para relacionar claves con valores, como si fuera una pequeña base de datos. Las tuplas y conjuntos también tienen sus propios usos, sobre todo cuando necesitamos datos que no cambien o evitar elementos repetidos. Entender cuándo usar cada una de estas estructuras me ha permitido escribir código más limpio y eficiente.

La modularidad es otro concepto que he aprendido a valorar. Organizar el código en funciones no solo lo hace más entendible, sino que también permite reutilizarlo. Crear funciones con parámetros y retornos me enseñó que un buen programa no es el que tiene muchas líneas, sino el que está bien distribuido y se puede leer como si fuera una historia bien contada. Además, conocer el ámbito local y global de las variables evita errores típicos cuando el programa crece y se vuelve más complejo.

5. Observaciones y Comentarios

Aprendí que comprender estos conceptos fundamentales de programación me permite escribir código más organizado y funcional. Me di cuenta de que cada tema, desde las estructuras de datos hasta la programación orientada a objetos, aporta claridad y estructura al desarrollo de un programa. También entendí que manejar errores, trabajar con archivos y aplicar la modularidad hace que mi trabajo sea más eficiente y profesional. En general, este aprendizaje me ha ayudado a ver la programación de una forma más completa y práctica.

6. Conclusión

En conclusión, todo lo que aprendí en este proceso me ayudó a entender mejor cómo funciona realmente la programación. Cada tema, desde la sintaxis hasta la programación orientada a objetos, me permitió ver que el código puede ser más ordenado, claro y fácil de manejar cuando se aplican bien estos conceptos. También comprendí lo útil que son las estructuras de datos, la modularidad y el manejo de errores para crear programas que funcionen de verdad. Siento que con todo esto tengo una base más sólida y más confianza para seguir aprendiendo y mejorando.

7. Bibliografías traducido al español

Gaddis, T. (2018). *Starting out with Python*. Pearson.

Guttag, J. V. (2016). *Introduction to computation and programming using Python*. MIT Press.

Downey, A. B. (2015). *Think Python: How to think like a computer scientist*. O'Reilly Media.

Lafore, R. (2012). *Object-oriented programming in C++*. Sams Publishing.